

Resultados de Pruebas Unitarias y Cobertura - SmartLogix

1. Objetivo

Demostrar buenas practicas de desarrollo mediante pruebas unitarias en frontend y backend, junto con reportes de cobertura de codigo.

2. Comandos de ejecucion

Backend

Ejecutar en cada modulo:

```
mvn clean test jacoco:report
```

Modulos:

- bff
- ms-inventario
- ms-pedidos
- ms-envios

Frontend

```
cd frontend
npm install
npm run test:coverage
```

Script unificado

```
bash scripts/generar-reportes-pruebas.sh
```

3. Pruebas implementadas

Componente	Archivo de prueba	Cantidad aproximada	Enfoque
BFF	BffServiceTest.java	8	Dashboard agregado, degradacion controlada, proxies CRUD y cambios de estado
Inventario	InventarioServiceTest.java	9	Listar, buscar por SKU, crear, validar duplicados, reducir stock, eliminar
Pedidos	PedidoServiceTest.java	8	Obtener, listar, cambiar estado, Factory Method, filtro por estado
Envios	EnvioServiceTest.java	7	Listar, obtener, crear envio, actualizar estado, fecha de entrega, filtro por pedido
Frontend	StatCard.test.js y useDashboard.test.js	10	Renderizado de componentes, accesibilidad, carga de datos y errores

Total aproximado: 42 pruebas unitarias.

4. Ubicacion de reportes

Componente	Reporte esperado
BFF	bff/target/site/jacoco/index.html

Componente	Reporte esperado
Inventario	ms-inventario/target/site/jacoco/index.html
Pedidos	ms-pedidos/target/site/jacoco/index.html
Envios	ms-envios/target/site/jacoco/index.html
Frontend	frontend/coverage/lcov-report/index.html

5. Tabla para registrar cobertura final

Completar esta tabla despues de ejecutar los comandos en un equipo con Maven y Node instalados.

Componente	Pruebas	Resultado	Cobertura lineas	Evidencia
BFF	8	Pendiente de ejecutar	Pendiente	JaCoCo
ms-inventario	9	Pendiente de ejecutar	Pendiente	JaCoCo
ms-pedidos	8	Pendiente de ejecutar	Pendiente	JaCoCo
ms-envios	7	Pendiente de ejecutar	Pendiente	JaCoCo
Frontend	10	Pendiente de ejecutar	Pendiente	React coverage

6. Buenas practicas aplicadas

- Separacion por capas en backend.
- DTO para contratos de API.
- Repositorios JPA mockeables en pruebas.
- Servicios con reglas de negocio aisladas.
- BFF con degradacion controlada ante fallos.
- Frontend con Custom Hooks y Service Layer.
- Pruebas unitarias por componente.
- Configuracion de JaCoCo y coverage NPM.